End-to-End DevOps Project

Terraform · Jenkins · SonarQube · Docker · CloudWatch · SNS

Submitted by: Prajwal V

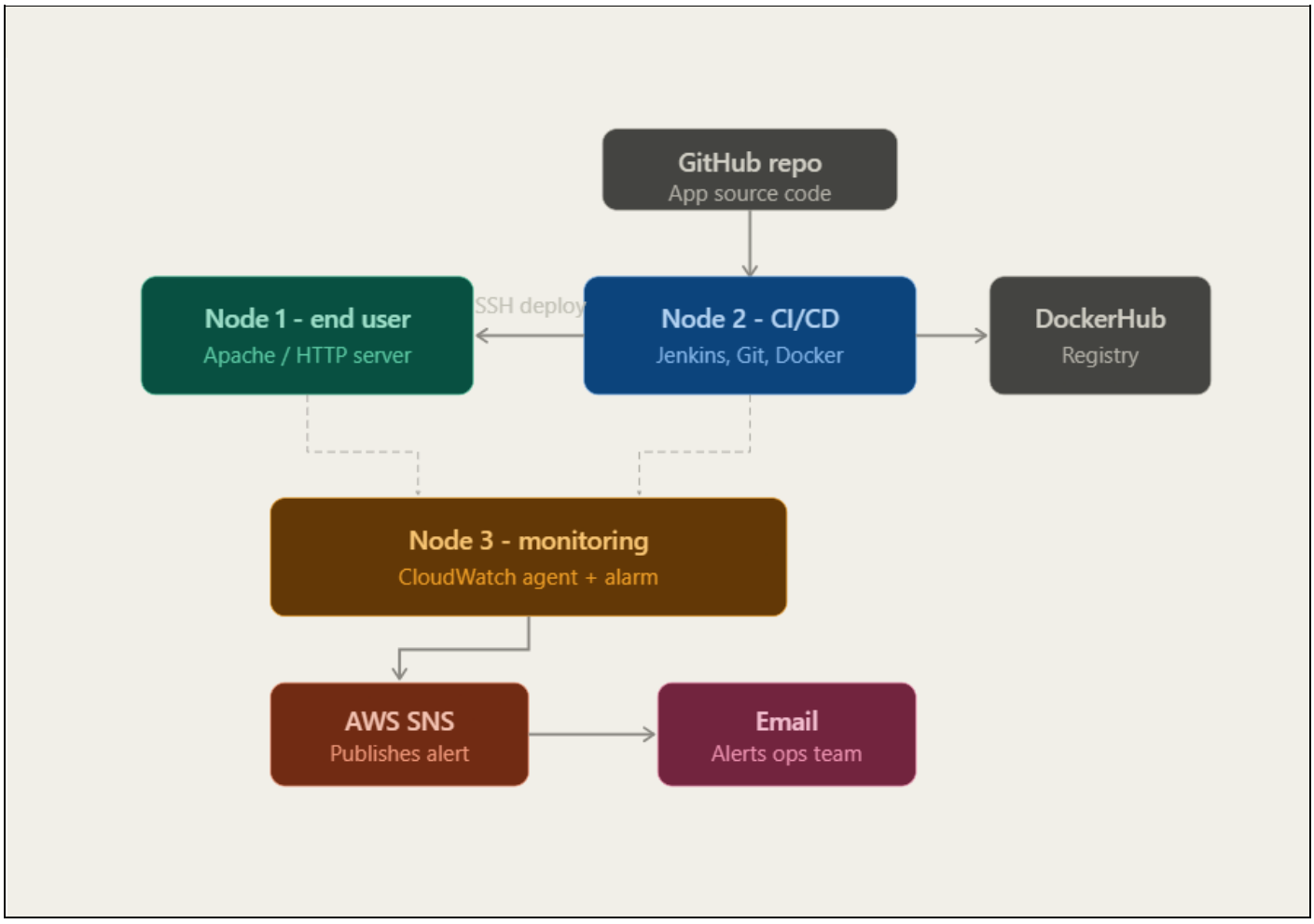
Mail: **hprajwalv@gmail**
Date: 13 September 2026
Batch : Cloud and DevOps

1. Project overview

This project implements a complete DevOps workflow on AWS, combining infrastructure automation with a full CI/CD pipeline. AWS resources are provisioned as code using Terraform across a 3-node architecture: Node 1 hosts the deployed application behind Apache, Node 2 runs the CI/CD toolchain (Jenkins, SonarQube, Docker, Trivy), and Node 3 handles monitoring via AWS CloudWatch with SNS email alerting. The pipeline pulls application code from GitHub, runs static code analysis and a quality gate through SonarQube, performs dependency and image vulnerability scanning with Trivy, builds and pushes a Docker image to DockerHub, and deploys the container to Node 1 over SSH. Node 3 continuously tracks CPU utilization and other metrics and triggers an email alert through SNS if a defined threshold is breached.

2. Architecture diagram

The diagram below shows the 3-node architecture: source code flows from GitHub into Node 2, which builds, scans, and pushes a Docker image to DockerHub and deploys it to Node 1 over SSH. Node 3 monitors both nodes via CloudWatch and raises alerts through SNS.



Node responsibilities

Node	Role
Node 1 - End user	Apache/HTTP server; serves the deployed application to the public on port 80
Node 2 - CI/CD	Jenkins, Git, SonarQube, Docker, Trivy, NodeJS; builds, scans, and pushes the app
Node 3 - Monitoring	CloudWatch agent + CPU alarm; SNS topic with email subscription for alerts

3. Step 1 - Provision AWS resources with Terraform

AWS infrastructure is provisioned using Terraform, split across standard configuration files:

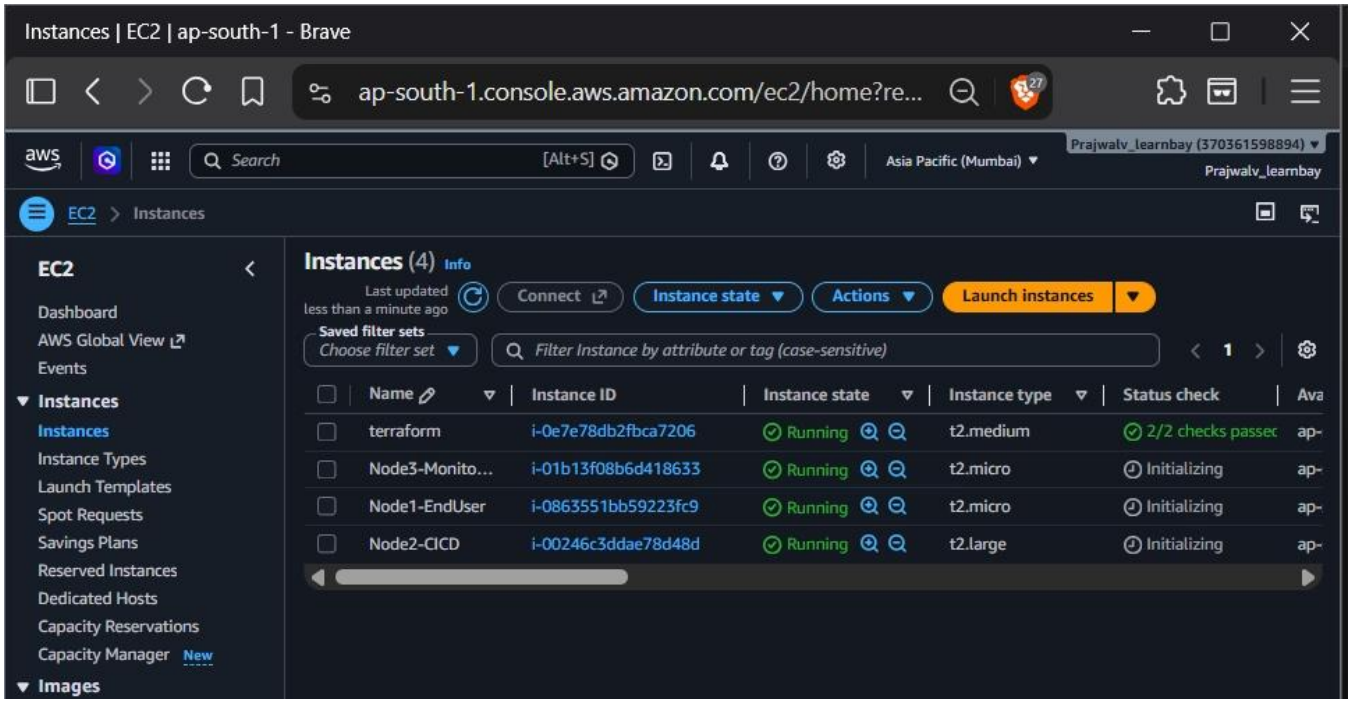
File	Purpose
main.tf	Terraform configuration block
provider.tf	AWS provider and region configuration
resource.tf	EC2 instances (x3), VPC, security groups, IAM roles, key pairs
outputs.tf	Parameterized values and outputs such as each node's public IP

Security groups

- Node 1: inbound 80 (HTTP, from anywhere), inbound 22 (SSH, from Node 2's security group)
- Node 2: inbound 22 (SSH), 8080 (Jenkins), 9000 (SonarQube)
- Node 3: inbound 22 (SSH) only - communicates outward to CloudWatch

Execution flow

```
terraform init      # initialize provider plugins
terraform plan      # preview infrastructure changes
terraform apply -auto-approve   # provision EC2 & networking
terraform destroy   # teardown resources (cleanup)
```



4. Node 1 - End user (Apache)

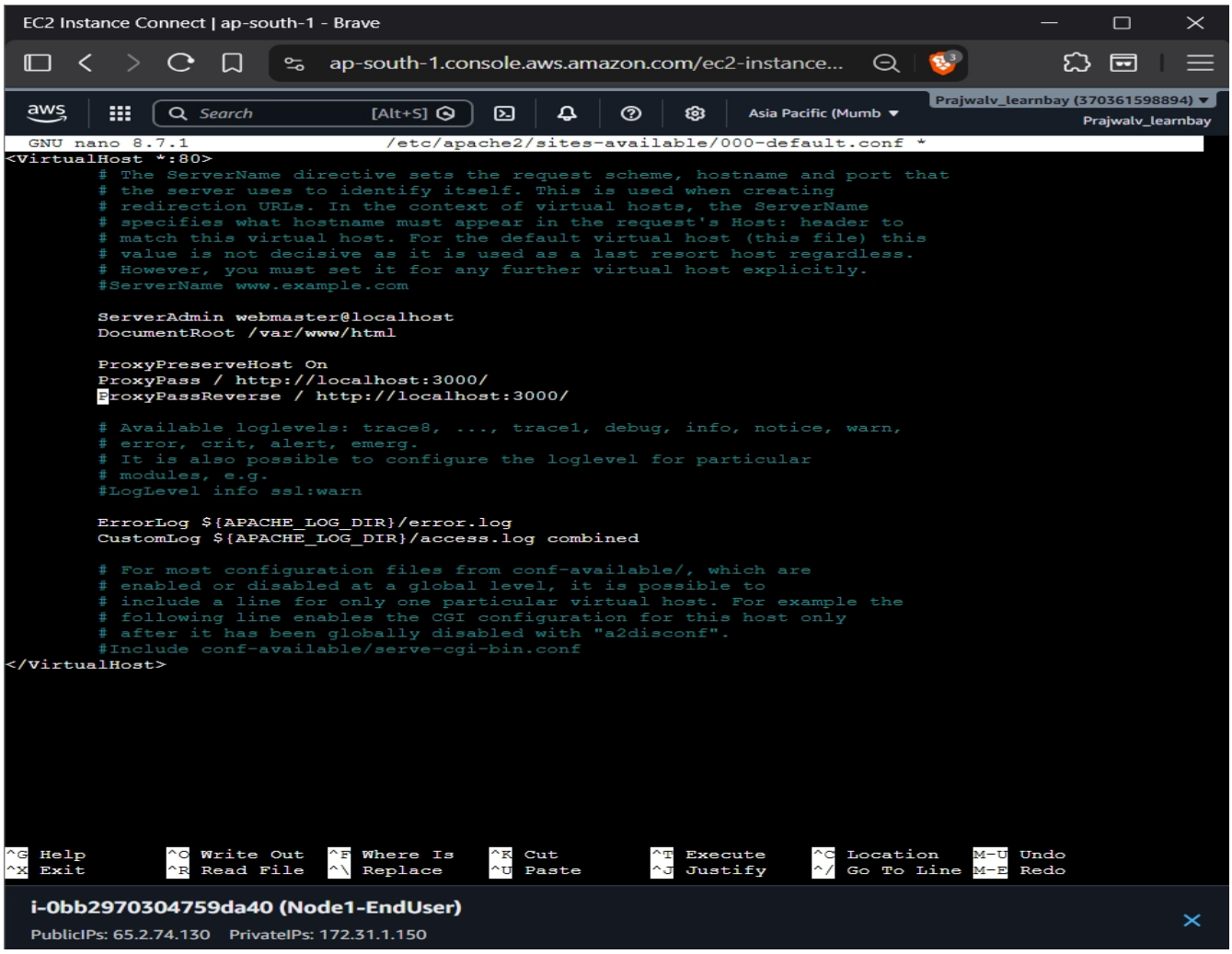
Node 1 runs Apache to serve the application to the public. Apache and Docker are both installed on this instance: Docker runs the deployed application container on port 3000, and Apache is configured as a reverse proxy (using the proxy and proxy_http modules) forwarding incoming requests on port 80 to localhost:3000. This means Apache's configuration never changes between deployments - only the container behind it is replaced each time Jenkins deploys a new build

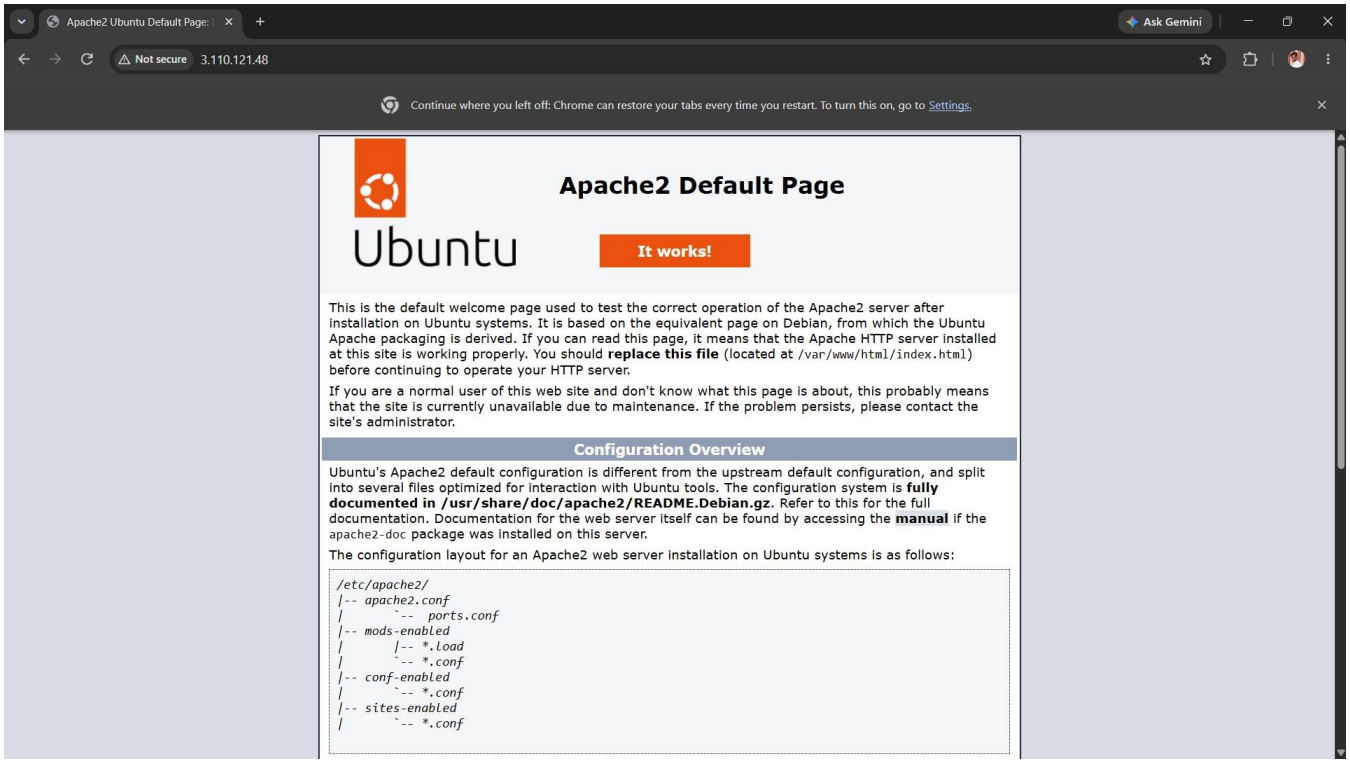
Made the modification to `/etc/apache2/sites-available/000-default.conf` file. inside the file under <VirtualHost *:80> block added

ProxyPreserveHost On

ProxyPass / http://localhost:3000/

ProxyPassReverse / http://localhost:3000/

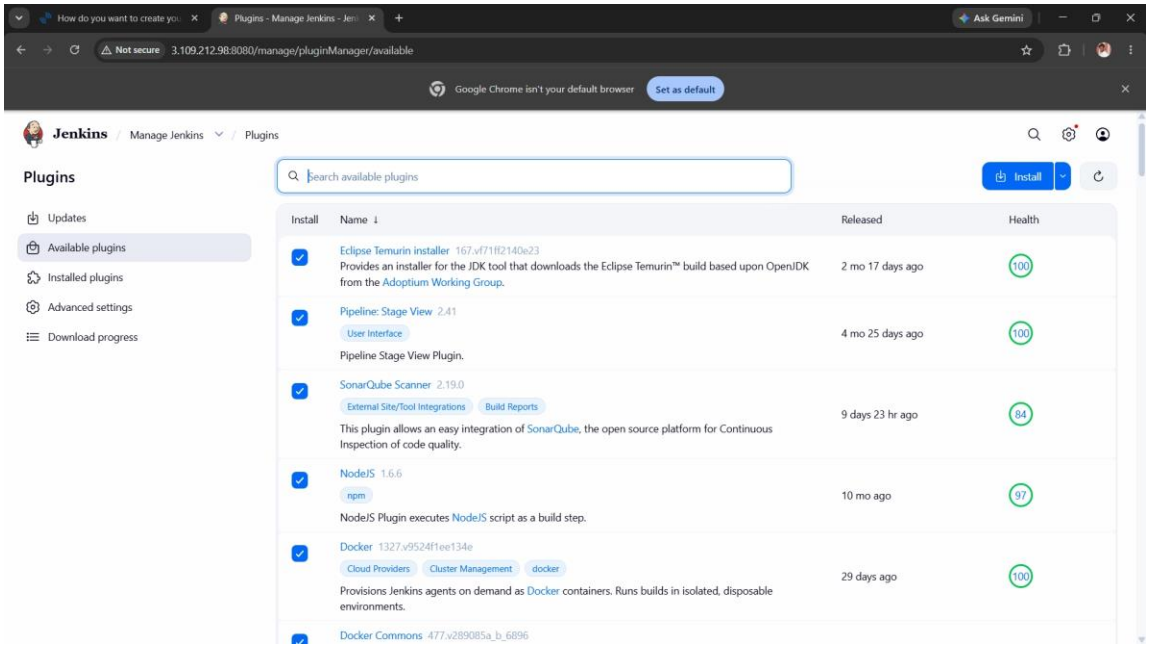

EC2 Instance Connect | ap-south-1 - Brave
ap-south-1.console.aws.amazon.com/ec2-instance...
Search [Alt+S]
Asia Pacific (Mumb
Prajwalv_learnbay (370361598894)
Prajwalv_learnbay
GNU nano 8.7.1 /etc/apache2/sites-available/000-default.conf *
<VirtualHost *:80>
The ServerName directive sets the request scheme, hostname and port that
the server uses to identify itself. This is used when creating
redirection URLs. In the context of virtual hosts, the ServerName
specifies what hostname must appear in the request's Host: header to
match this virtual host. For the default virtual host (this file) this
value is not decisive as it is used as a last resort host regardless.
However, you must set it for any further virtual host explicitly.
#ServerName www.example.com
ServerAdmin webmaster@localhost
DocumentRoot /var/www/html
ProxyPreserveHost On
ProxyPass / http://localhost:3000/
ProxyPassReverse / http://localhost:3000/
Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
error, crit, alert, emerg.
It is also possible to configure the loglevel for particular
modules, e.g.
#LogLevel info ssl:warn
ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined
For most configuration files from conf-available/, which are
enabled or disabled at a global level, it is possible to
include a line for only one particular virtual host. For example the
following line enables the CGI configuration for this host only
after it has been globally disabled with "a2disconf".
#Include conf-available/serve-cgi-bin.conf
</VirtualHost>
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo
i-0bb2970304759da40 (Node1-EndUser)
PublicIPs: 65.2.74.130 PrivateIPs: 172.31.1.150


Apache2 Ubuntu Default Page:
Ask Gemini
Not secure 3.110.121.48
Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to Settings.
Apache2 Default Page
Ubuntu
It works!
This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should replace this file (located at /var/www/html/index.html) before continuing to operate your HTTP server.
If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.
Configuration Overview
Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is fully documented in /usr/share/doc/apache2/README.Debian.gz. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the manual if the apache2-doc package was installed on this server.
The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:
/etc/apache2/
|-- apache2.conf
| `-- ports.conf
|-- mods-enabled
| |-- *.load
| `-- *.conf
|-- conf-enabled
| `-- *.conf
|-- sites-enabled
| `-- *.conf

5. Node 2 - CI/CD (Jenkins, SonarQube, Docker)

Tools installed

- Jenkins - pipeline orchestration (accessed on port 8080)
- SonarQube - static code analysis (accessed on port 9000)
- Docker (Common, Pipeline, API) - image build and push
- Trivy - filesystem and image vulnerability scanning
- NodeJS - installs and builds JavaScript/Node app dependencies
- Eclipse Temurin (JDK) - required by Jenkins and the SonarQube scanner
- Pipeline Stage View - visualizes the CI/CD pipeline stages

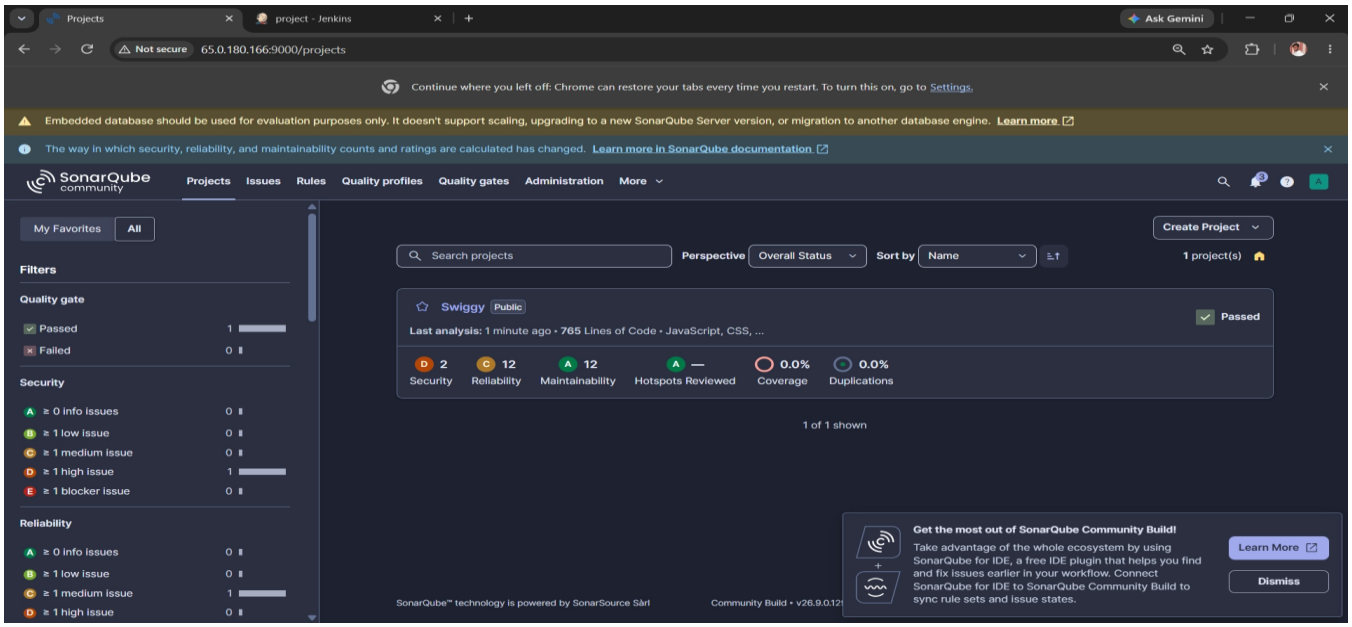


Global tool configuration

Tool	Configured as
JDK	jdk17 (Eclipse Temurin, adoptium.net)
SonarQube Scanner	sonar-scanner
NodeJS	node20
Docker	docker (latest version)

SonarQube integration

A SonarQube token was generated under Administration > Security > Users and stored in Jenkins as a Secret Text credential with ID sonar-token. A webhook was configured in SonarQube pointing to http://<Node2-IP>:8080/sonarqube-webhook/ so SonarQube notifies Jenkins of scan results, allowing the pipeline's Quality Gate stage to pass or fail accordingly. The SonarQube server was registered in Jenkins under Configure System as sonar-server.



DockerHub integration

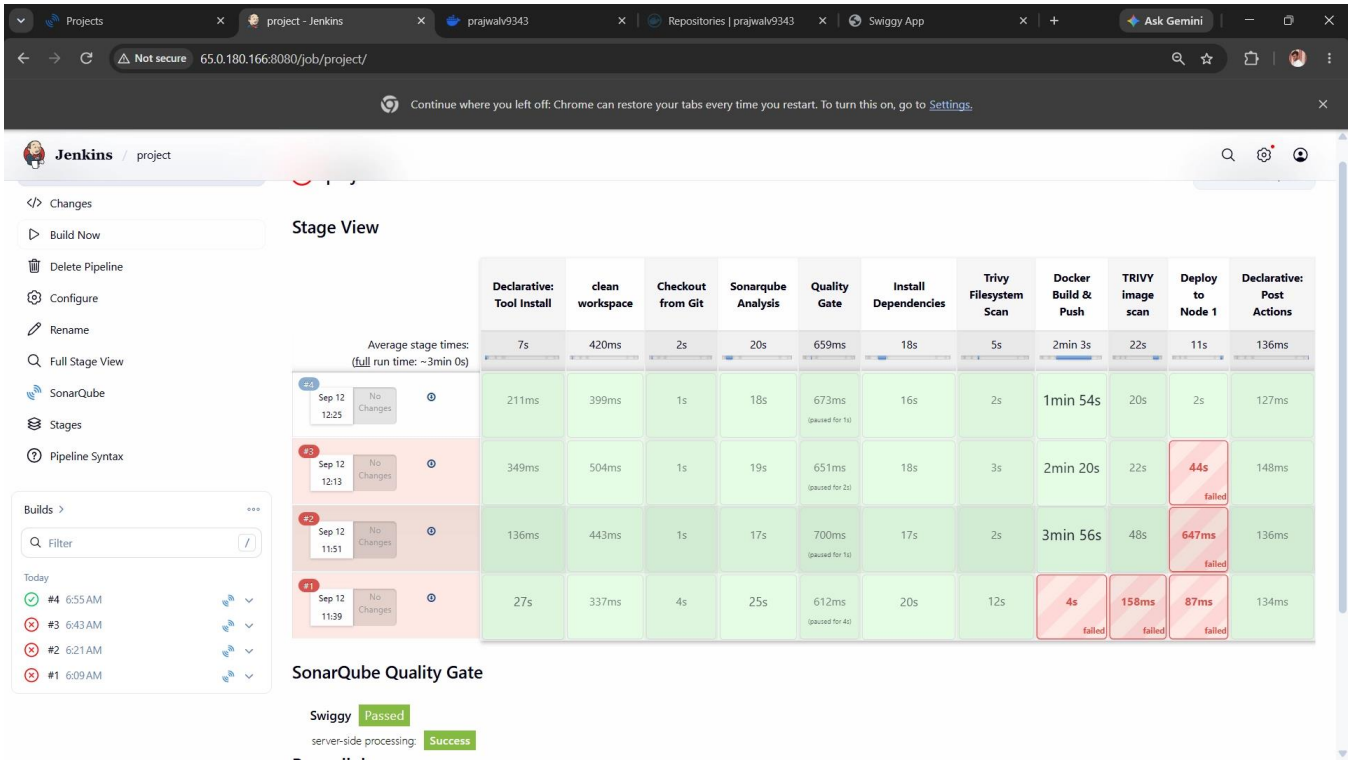
DockerHub credentials were added in Jenkins as a Username/Password credential with ID docker-creds, allowing the pipeline to authenticate, build, tag, and push the Docker image via withDockerRegistry.

6. Jenkins pipeline

The Jenkins pipeline (Jenkinsfile) runs the following stages in order:

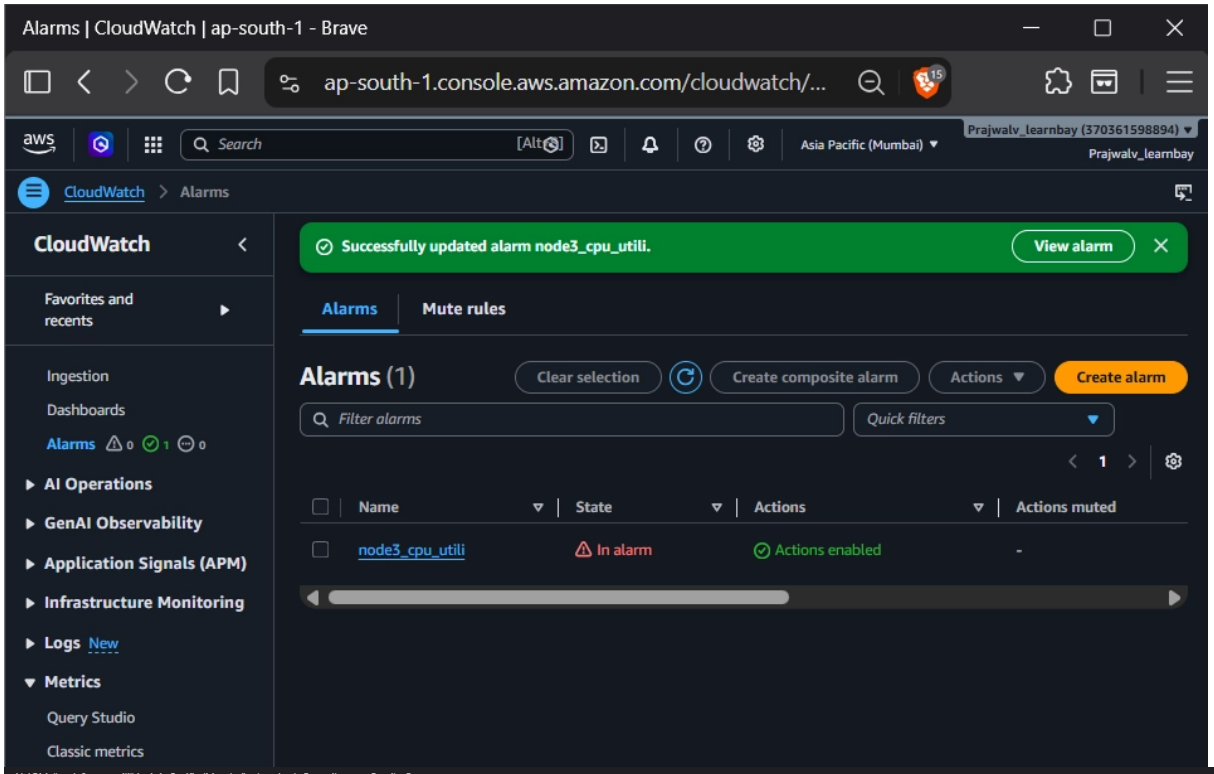
Stage	Purpose
Clean Workspace	Wipes any leftover files from the previous build
Checkout from Git	Pulls the latest application code from the GitHub repository
SonarQube Analysis	Runs sonar-scanner and uploads results to the SonarQube server
Quality Gate	Waits (up to a 2-minute timeout) for SonarQube's pass/fail result
Install Dependencies	Runs npm install for the application's Node.js dependencies
Trivy Filesystem Scan	Scans source code and dependencies for HIGH/CRITICAL CVEs
Docker Build & Push	Builds the Docker image, tags it, and pushes it to DockerHub
Trivy Image Scan	Scans the built Docker image for vulnerabilities
Deploy to Node 1	SSHes into Node 1 and runs the latest container to replace the old one

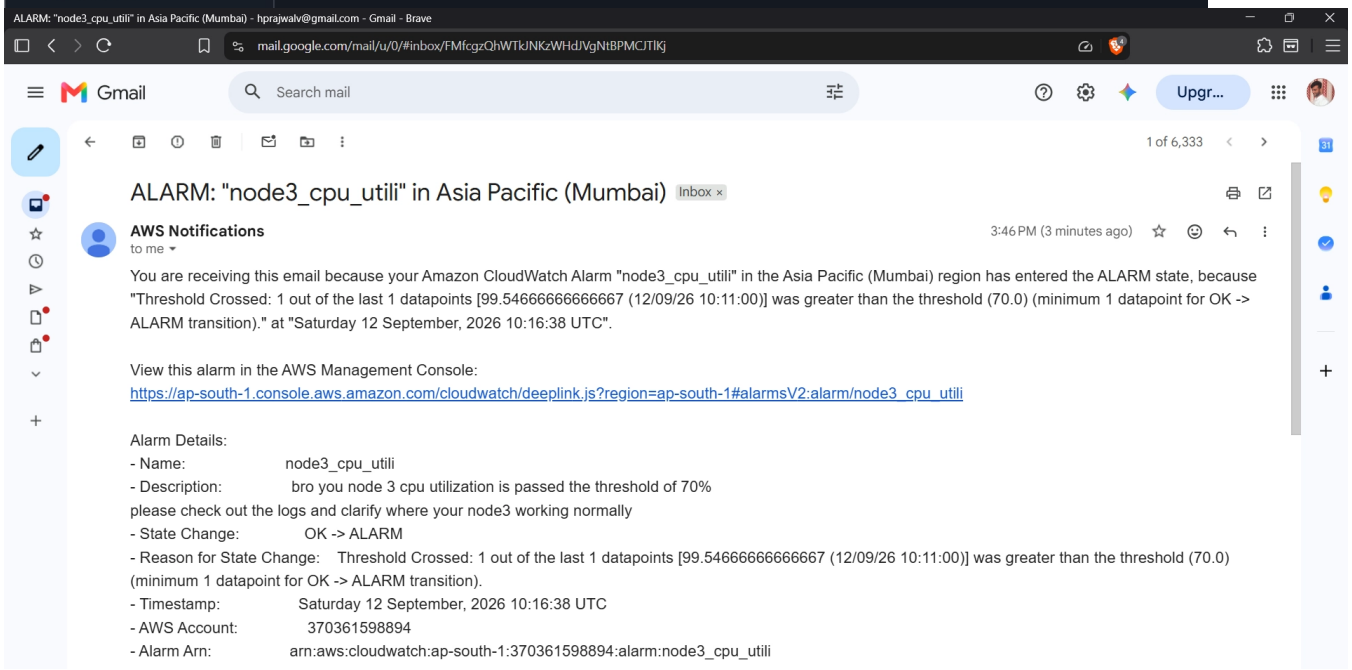
The original single-node version of this pipeline (from the data set) ran the final deploy step locally with a plain docker run. In this 3-node version, that stage is replaced with an SSH command from Node 2 into Node 1, using the Jenkins SSH Agent plugin and a stored SSH credential, so the container actually runs on the public-facing Node 1 rather than on the CI/CD box.



7. Node 3 - Monitoring (CloudWatch + SNS)

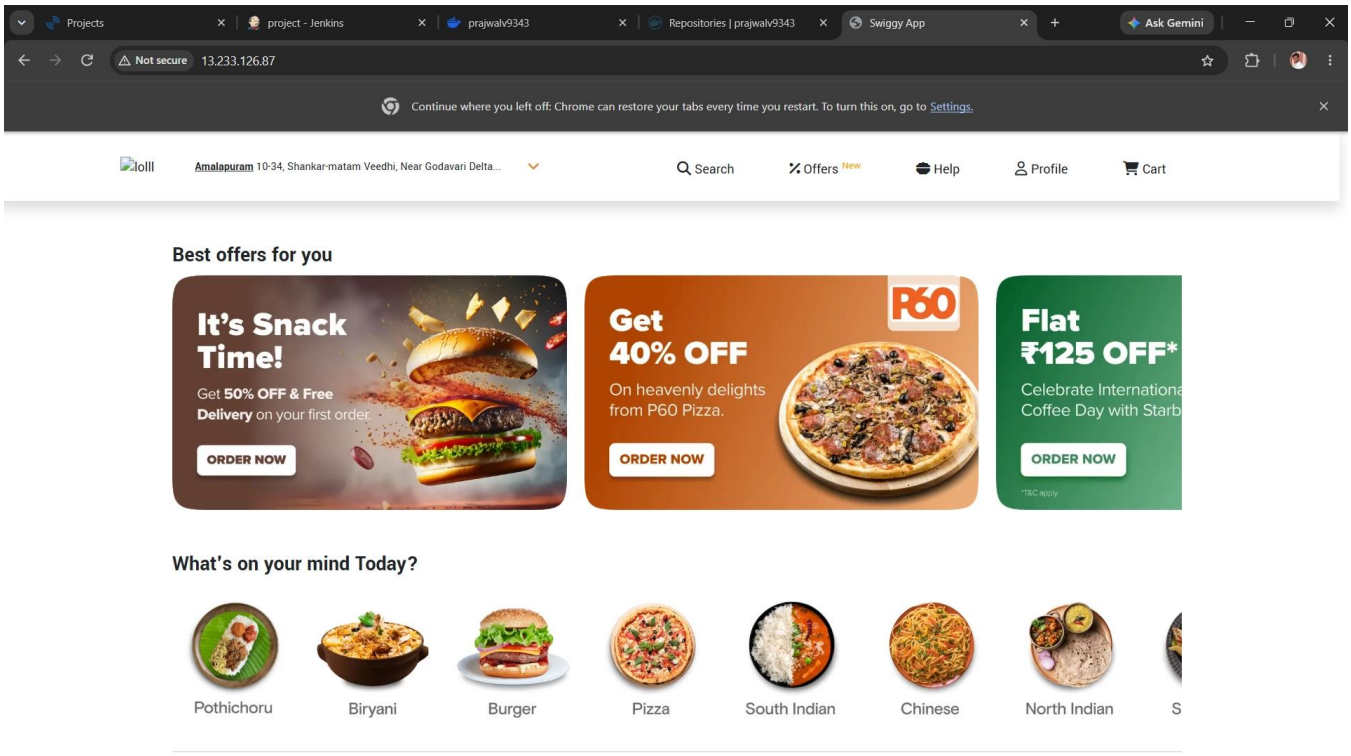
Node 3 hosts the monitoring and alerting configuration for the project. An IAM role with the CloudWatchAgentServerPolicy attached was assigned to the monitored instance(s), and the CloudWatch agent was installed to collect metrics such as CPU utilization, memory, and disk usage. A CloudWatch alarm was configured on CPUUtilization with a threshold (e.g. greater than 70% sustained) that, when breached, publishes a notification to an SNS topic. An email address was subscribed to the SNS topic and confirmed, so that any alarm triggers an email notification to the operations contact.





8. Final result

The completed pipeline deploys the application to Node 1, accessible at http://<Node1-Public-IP>. A successful run confirms infrastructure provisioning, code quality and security checks, containerized build and push, and live deployment all functioning together as one workflow.



9. Challenges faced and learnings

1. Disk full during SonarQube image pull (no space left on device)

Docker failed mid-extraction pulling the SonarQube image because the EC2 root volume was too small. Diagnosed with `df -h`, then resolved by live-resizing the EBS volume in the AWS console and extending the filesystem on the running instance with `growpart/resize2fs` — without any downtime.

2. SonarQube container killed by out-of-memory (OOM)

Confirmed via `dmesg | grep -i "killed process"` that the Linux kernel's OOM killer terminated SonarQube's JVM process mid-pipeline, since Jenkins + SonarQube + an active build were all competing for RAM on the same instance. Fixed by explicitly capping SonarQube's JVM heap sizes (SONAR_WEB_JAVAOPTS, etc.) and adding swap space as a buffer.

3. Docker login failure from a stale bundled Docker client

Jenkins' "Install automatically" Docker tool option downloaded a years-old static Docker client binary (API v1.29) that was incompatible with the modern Docker daemon (requiring v1.44+) already installed on the host — causing `docker login` to fail with a version mismatch, even though credentials were correct.

Key learning from this project

The important skill this project reinforced wasn't any single tool, but a debugging habit: always check logs and system state (`journalctl`, `dmesg`, `docker ps -a`, `df -h`, `free -h`) before assuming a configuration is wrong. Nearly every failure I hit — the disk-full error, the OOM-killed SonarQube container, the stale Docker client — looked like a tooling misconfiguration at first, but was actually a resource or environment issue that logs immediately revealed once I knew where to look.

10. Conclusion

This project demonstrates a complete DevOps workflow across a 3-node AWS architecture: infrastructure as code with Terraform, CI/CD automation with Jenkins, code quality and security scanning with SonarQube and Trivy, containerized deployment with Docker, and proactive monitoring and alerting with CloudWatch and SNS - covering the full path from source code to a live, monitored production deployment.